

Advanced SQL Queries and Execution Plans

Aim

To perform advanced SQL queries on the Employee–Department–Project database using INNER JOIN, LEFT JOIN, SELF JOIN, multi-table joins, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT operations, and to compare query execution plans using EXPLAIN.

Software / Requirements

- MySQL / MySQL Workbench
- CompanyDB database created in Experiment 1
- Employee, Department, Project, and Employee_Project tables

Theory

SQL joins combine rows from two or more tables based on related columns. INNER JOIN returns matching rows, while LEFT JOIN preserves all rows from the left table. A SELF JOIN joins a table with itself. Subqueries allow one query to use the result of another query, while correlated subqueries refer to values from the outer query. EXISTS tests whether a subquery returns at least one row. EXPLAIN displays the execution plan selected by the database optimizer, including access methods, indexes, estimated rows, and additional operations.

Database Used

The queries in this experiment use the CompanyDB schema created in Experiment 1. The schema contains Department, Employee, Project, and Employee_Project tables.

1. INNER JOIN

Display employees along with the departments to which they belong.

```
SELECT e.Emp_ID,  
       e.Emp_Name,  
       e.Job_Title,  
       d.Dept_Name  
FROM Employee e  
INNER JOIN Department d  
ON e.Dept_ID = d.Dept_ID;
```

INNER JOIN returns only those employees for whom a matching department exists.

2. LEFT JOIN

Display all departments and their employees, including departments with no matching employee.

```
SELECT d.Dept_ID,  
       d.Dept_Name,  
       e.Emp_Name  
FROM Department d  
LEFT JOIN Employee e  
ON d.Dept_ID = e.Dept_ID;
```

LEFT JOIN preserves every row from the left table, Department, even when no matching employee exists.

3. SELF JOIN

Find pairs of employees who work in the same department.

```
SELECT e1.Emp_Name AS Employee_1,  
       e2.Emp_Name AS Employee_2,  
       e1.Dept_ID  
FROM Employee e1  
INNER JOIN Employee e2  
ON e1.Dept_ID = e2.Dept_ID  
AND e1.Emp_ID < e2.Emp_ID;
```

The condition `e1.Emp_ID < e2.Emp_ID` prevents duplicate pairs and prevents an employee from being paired with themselves.

4. Multi-Table / 3-Way Join

Display employee name, department name, and project name. Because Employee and Project have a many-to-many relationship, the Employee_Project bridge table is required. Therefore, the physical query joins four tables.

```
SELECT e.Emp_Name,  
       d.Dept_Name,  
       p.Project_Name  
FROM Employee e  
INNER JOIN Department d  
ON e.Dept_ID = d.Dept_ID  
INNER JOIN Employee_Project ep  
ON e.Emp_ID = ep.Emp_ID  
INNER JOIN Project p  
ON ep.Project_ID = p.Project_ID;
```

5. Correlated Subquery

Find employees whose salary is greater than the average salary of their own department.

```
SELECT e.Emp_ID,  
       e.Emp_Name,  
       e.Salary,  
       e.Dept_ID  
FROM Employee e  
WHERE e.Salary > (  
    SELECT AVG(e2.Salary)  
    FROM Employee e2  
    WHERE e2.Dept_ID = e.Dept_ID  
);
```

The inner query is correlated because it refers to `e.Dept_ID` from the outer Employee query. The department average is evaluated in relation to the current employee's department.

6. EXISTS

Find departments that have at least one employee earning more than ₹100,000.

```
SELECT d.Dept_ID,  
       d.Dept_Name
```

```

FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.Dept_ID = d.Dept_ID
        AND e.Salary > 100000
);

```

EXISTS evaluates to TRUE when the subquery returns at least one matching row.

7. Simulated INTERSECT

Find employees who earn more than ₹80,000 and belong to either the AI or Data Science department. The query demonstrates the intersection of two result sets using INNER JOIN.

```

SELECT a.Emp_ID,
       a.Emp_Name
FROM
(
    SELECT Emp_ID, Emp_Name
    FROM Employee
    WHERE Salary > 80000
) a
INNER JOIN
(
    SELECT Emp_ID, Emp_Name
    FROM Employee
    WHERE Dept_ID IN (1, 4)
) b
ON a.Emp_ID = b.Emp_ID;

```

The INNER JOIN keeps only employee IDs appearing in both result sets, thereby simulating INTERSECT.

8. Simulated EXCEPT

Find employees who earn more than ₹80,000 but are not in the Cyber Security department.

```

SELECT e.Emp_ID,
       e.Emp_Name
FROM Employee e
WHERE e.Salary > 80000
AND NOT EXISTS (
    SELECT 1
    FROM Employee e2
    WHERE e2.Emp_ID = e.Emp_ID
        AND e2.Dept_ID = 3
);

```

NOT EXISTS removes rows for which a matching Cyber Security employee record is found, thereby simulating an EXCEPT operation.

9. EXPLAIN — Execution Plan

EXPLAIN displays how MySQL plans to execute a SELECT statement. Important columns include type, possible_keys, key, rows, and Extra. The actual execution plan should be recorded from the student's MySQL environment because it can vary with indexes, statistics, data volume, and MySQL version.

Query 1: EXPLAIN for a simple selection

```
EXPLAIN
SELECT *
FROM Employee
WHERE Salary > 90000;
```

Query 2: EXPLAIN for INNER JOIN

```
EXPLAIN
SELECT e.Emp_Name,
       d.Dept_Name
FROM Employee e
INNER JOIN Department d
ON e.Dept_ID = d.Dept_ID;
```

Query 3: EXPLAIN for correlated subquery

```
EXPLAIN
SELECT e.Emp_ID,
       e.Emp_Name,
       e.Salary
FROM Employee e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employee e2
    WHERE e2.Dept_ID = e.Dept_ID
);
```

Query 4: EXPLAIN for EXISTS

```
EXPLAIN
SELECT d.Dept_Name
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.Dept_ID = d.Dept_ID
    AND e.Salary > 100000
);
```

13. Output / Screenshots

Emp_ID	Emp_Name	Job_Title	Dept_Name
101	Aarav Sharma	AI Engineer	Artificial Intelligence
102	Priya Verma	ML Engineer	Artificial Intelligence
103	Rohan Gupta	AI Developer	Artificial Intelligence
104	Ananya Singh	Research Engineer	Artificial Intelligence
105	Karan Mehta	ML Engineer	Artificial Intelligence
106	Neha Kapoor	AI Analyst	Artificial Intelligence
107	Rahul Kumar	Software Engineer	Software Development
108	Sneha Patel	Backend Developer	Software Development
109	Aditya Jain	Frontend Developer	Software Development
110	Simran Kaur	Full Stack Developer	Software Development
111	Vivek Mishra	Software Engineer	Software Development
112	Ishita Rao	QA Engineer	Software Development
113	Arjun Malhotra	Security Engineer	Cyber Security
114	Megha Joshi	Cyber Security Analyst	Cyber Security
115	Yash Thakur	Security Consultant	Cyber Security
116	Pooja Nair	SOC Analyst	Cyber Security

Dept_ID	Dept_Name	Emp_Name
1	Artificial Intelligence	Aarav Sharma
1	Artificial Intelligence	Priya Verma
1	Artificial Intelligence	Rohan Gupta
1	Artificial Intelligence	Ananya Singh
1	Artificial Intelligence	Karan Mehta
1	Artificial Intelligence	Neha Kapoor
2	Software Development	Rahul Kumar
2	Software Development	Sneha Patel
2	Software Development	Aditya Jain
2	Software Development	Simran Kaur
2	Software Development	Vivek Mishra
2	Software Development	Ishita Rao
3	Cyber Security	Arjun Malhotra
3	Cyber Security	Megha Joshi
3	Cyber Security	Yash Thakur
3	Cyber Security	Pooja Nair

Employee_1	Employee_2	Dept_ID
Aarav Sharma	Priya Verma	1
Aarav Sharma	Rohan Gupta	1
Aarav Sharma	Ananya Singh	1
Aarav Sharma	Karan Mehta	1
Aarav Sharma	Neha Kapoor	1
Priya Verma	Rohan Gupta	1
Priya Verma	Ananya Singh	1
Priya Verma	Karan Mehta	1
Priya Verma	Neha Kapoor	1
Rohan Gupta	Ananya Singh	1
Rohan Gupta	Karan Mehta	1
Rohan Gupta	Neha Kapoor	1
Ananya Singh	Karan Mehta	1
Ananya Singh	Neha Kapoor	1
Karan Mehta	Neha Kapoor	1
Rahul Kumar	Sneha Patel	2

Emp_Name	Dept_Name	Project_Name
Aarav Sharma	Artificial Intelligence	AI Recommendation System
Priya Verma	Artificial Intelligence	AI Recommendation System
Ananya Singh	Artificial Intelligence	AI Recommendation System
Rohan Gupta	Artificial Intelligence	Computer Vision Platform
Karan Mehta	Artificial Intelligence	Computer Vision Platform
Neha Kapoor	Artificial Intelligence	Computer Vision Platform
Rahul Kumar	Software Development	E-Commerce Platform
Sneha Patel	Software Development	E-Commerce Platform
Simran Kaur	Software Development	E-Commerce Platform
Aditya Jain	Software Development	Mobile Banking Application
Vivek Mishra	Software Development	Mobile Banking Application
Ishita Rao	Software Development	Mobile Banking Application
Arjun Malhotra	Cyber Security	Cyber Threat Detection
Megha Joshi	Cyber Security	Cyber Threat Detection
Dev Patel	Cyber Security	Cyber Threat Detection
Yash Thakur	Cyber Security	Security Monitoring System
Pooja Nair	Cyber Security	Security Monitoring System

Emp_ID	Emp_Name	Salary	Dept_ID
102	Priya Verma	92000.00	1
104	Ananya Singh	105000.00	1
105	Karan Mehta	88000.00	1
108	Sneha Patel	82000.00	2
110	Simran Kaur	95000.00	2
111	Vivek Mishra	78000.00	2
113	Arjun Malhotra	90000.00	3
115	Yash Thakur	98000.00	3
117	Dev Patel	87000.00	3
118	Tanvi Shah	91000.00	3
119	Akash Singh	100000.00	4
121	Mohit Agarwal	95000.00	4
123	Nikhil Bansal	89000.00	4
125	Manish Verma	85000.00	5
129	Varun Gupta	73000.00	5

Dept_ID	Dept_Name
1	Artificial Intelligence

Emp_ID	Emp_Name
101	Aarav Sharma
102	Priya Verma
104	Ananya Singh
105	Karan Mehta
119	Akash Singh
121	Mohit Agarwal
123	Nikhil Bansal

Emp_ID	Emp_Name
101	Aarav Sharma
102	Priya Verma
104	Ananya Singh
105	Karan Mehta
108	Sneha Patel
110	Simran Kaur
119	Akash Singh
121	Mohit Agarwal
123	Nikhil Bansal
125	Manish Verma

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	NULL	ALL	PRIMARY	NULL	NULL	NULL	5	100.00	NULL
1	SIMPLE	e	NULL	ref	Dept_ID	Dept_ID	5	sandbox_db.d.Dept_ID	15	100.00	NULL
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	e	NULL	ALL	NULL	NULL	NULL	NULL	30	100.00	Using index
2	DEPENDENT SUBQUERY	e2	NULL	ref	Dept_ID	Dept_ID	5	sandbox_db.e.Dept_ID	15	100.00	NULL
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	NULL	ALL	PRIMARY	NULL	NULL	NULL	5	100.00	Using index
1	SIMPLE	<subquery2>	NULL	eq_ref	<auto_distinct_key>	<auto_distinct_key>	5	sandbox_db.d.Dept_ID	1	100.00	Using index
2	MATERIALIZED	e	NULL	ALL	Dept_ID	NULL	NULL	NULL	30	100.00	Using index

11. Result

The Employee–Department–Project database was successfully queried using INNER JOIN, LEFT JOIN, SELF JOIN, multi-table joins, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT operations. Execution plans for selected queries were examined using EXPLAIN and their relevant plan characteristics were recorded for comparison.